COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	9/11/26	**DATE SUBMITTED:**	9/15/26
NAME:	SUBALISID, JAPPETH ROYCE B.	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

Hands-on Activity 8.1 Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of shell, merge and quick sort.

Preparation Task:

- Create an array of elements with random values. The array must contain 100 elements that are not sorted. Use the created array for each sorting algorithm below. Show the output of the preparation task on table 8-1 in section 6.
- Create a header file for the implementation of the different sorting algorithms.
- Import this header file into your main.cpp

A.1. Shell Sort

General Algorithm

The general algorithm for shell sort is given below.

```
shellSort(array, size)
        for interval i <- size/2n down to 1
                for each interval "i" in array
                        sort all the elements at interval "i"
end shellSort
```

```cpp
template<typename T>
void shellSort(T arr[], size_t arrSize) {
    for(int gap = arrSize/2; gap > 0; gap /= 2) {//gets a gap(half arr size), then halves again after loop
        for(int i = gap; i < arrSize; i++) {//iterates from gap to end
            T temp = arr[i]; //stores arr[gap] value
            int j = i; //gets gap index

            while(j >= gap && arr[j - gap] > temp) {//checks if gap and front needs swapping
                arr[j] = arr[j - gap]; //moves front into gap
                j -= gap; //sets j as front index, and stops loop
            }
            arr[j] = temp; //sets front as the gap value
        }
    }
}
```

```
Unsorted:
8 79 68 54 48 73 23 98 85 68 87 56 74 27 0 61 12 72 60 61 8 1 51 85 58 32 32 4 36 72 23 40 4 74 41 15 9 59 21 83 69 68 84 4
3 14 72 88 91 74 53 20 78 73 62 19 61 53 55 30 71 54 93 19 95 33 8 43 86 17 26 86 34 16 93 30 41 55 4 37 26 45 94 11 53 2 3
6 87 57 11 48 40 99 20 5 64 44 12 61 52 27
Shell sorted:
0 1 2 4 4 4 5 8 8 8 9 11 11 12 12 14 15 16 17 19 19 20 20 21 23 23 26 26 27 27 30 30 32 32 33 34 36 36 37 40 40 41 41 43 43
 44 45 48 48 51 52 53 53 53 54 54 55 55 56 57 58 59 60 61 61 61 61 62 64 68 68 68 69 71 72 72 72 73 73 74 74 74 78 79 83 84
 85 85 86 86 87 87 88 91 93 93 94 95 98 99
```

A.2. Merge Sort

General Algorithm

The general pseudo-code for the merge sort technique is given below.

Declare an array Arr of length N
If N=1, Arr is already sorted
If N>1,
Left = 0, right = N-1
Find middle = (left + right)/2
Call merge_sort(Arr,left,middle) =>sort first half recursively
Call merge_sort(Arr,middle+1,right) => sort second half recursively
Call merge(Arr, left, middle, right) to merge sorted arrays in above steps.
Exit

```
Unsorted:
42 93 7 11 30 8 95 87 71 75 84 14 24 32 38 67 93 27 1 82 74 81 75 22 42 90 53 14 12 41 33 79 55 43 45 77 80 52 38 62 40 24 4
4 79 78 46 1 45 55 13 84 83 9 45 63 55 65 41 46 86 48 84 18 76 44 96 90 75 10 69 9 91 87 0 48 38 15 96 9 0 81 69 61 18 9 62
50 54 34 10 55 38 96 43 16 74 72 83 77 55
Merge sorted:
0 0 1 1 7 8 9 9 9 9 10 10 11 12 13 14 14 15 16 18 18 22 24 24 27 30 32 33 34 38 38 38 38 40 41 41 42 42 43 43 44 44 45 45 45
 46 46 48 48 50 52 53 54 55 55 55 55 55 61 62 62 63 65 67 69 69 71 72 74 74 75 75 75 76 77 77 78 79 79 80 81 81 82 83 83 84
84 84 86 87 87 90 90 91 93 93 95 96 96 96
```

```cpp
template<typename T>
void merge(T arr[], int left, int middle, int right) {
    int i = left, j = middle + 1, k = left;
    T* temp = new T[right + 1]; //temp array with size of right+1

    while(i <= middle && j <= right) { //merging element in order
        if(arr[i] <= arr[j]) {
            temp[k++] = arr[i++]; //if sorted, copy, move forward
        } else {
            temp[k++] = arr[j++]; //if not sorted, get the other sub-array's first element
        }
    }

    while (i <= middle) { //insert all remaining left elements
        temp[k++] = arr[i++];
    }

    while (j <= right) { //insert all remaining right elements
        temp[k++] = arr[j++];
    }

    for(int l = left; l <= right; l++) { //copy the sorted temp array
        arr[l] = temp[l];
    }

    delete[] temp; //clear temp memory

}
```

```cpp
template<typename T>
void mergeSorter(T arr[], int left, int right){
    if(left < right) {
        int middle = (right + left) / 2;

        mergeSorter(arr, left, middle); //sorting first half first recursively
        mergeSorter(arr, middle + 1, right); //sorting second half next recursively
        merge(arr, left, middle, right); //merge sorted halves
    }
}

template<typename T>
void mergeSort(T arr[], size_t arrSize) {
    if(arrSize <= 1) {
        std::cout << "Array is already sorted./n";
    } else {
        mergeSorter(arr, 0, arrSize - 1);
    }
}
```

A.3. Quick Sort

```
quicksort(A, low, high)
begin
Declare array A[N] to be sorted
        low = 1st element; high = last element; pivot
if(low < high)
        begin
        pivot = partition (A,low,high);
        quicksort(A,low,pivot-1)
        quicksort(A,pivot+1,high)
        End
end
```

```cpp
template<typename T>
int partition(T arr[], int left, int right){
    T pivot = arr[right]; //gets pivot value
    int i = left - 1; //i to -1 or before first element of unsorted part of array

    for (int j = left; j < right; j++) {//go through array
        if (arr[j] <= pivot) { //if j < pivot,
            i++; //increment,
            std::swap(arr[i], arr[j]); //then swap(i,j)
        }
    }
    std::swap(arr[i + 1], arr[right]); //after j reaches pivot, swap pivot to i+1th element
    return ++i; //give the pivot's position
}

template<typename T>
void quickSorter(T arr[], int left, int right) {
    if(left < right) {
        int pivotIndex = partition(arr, left, right); //first gets the pivotIndex's proper place
        quickSorter(arr, left, pivotIndex - 1); //repeats process on the left side of the initial correct pivotIndex
        quickSorter(arr, pivotIndex + 1, right); // repeats process until the whole right side is also sorted
    }
}

template<typename T>
void quickSort(T arr[], size_t arrSize) {
   if(arrSize <= 1) {
        std::cout << "Array is already sorted./n";
    } else {
        quickSorter(arr, 0, arrSize - 1);
    }
}
```

```
Unsorted:
97 1 88 73 35 13 90 35 55 46 47 50 53 93 90 50 13 2 26 98 47 79 83 5 94 38 19 65 99 12 31 45 32 85 36 77 88 10 53 42 88 54 8
5 96 79 60 29 50 26 73 52 81 42 5 66 33 62 55 2 43 64 53 98 8 24 20 58 52 95 32 41 35 97 36 93 96 53 86 86 19 75 52 61 8 92
81 81 88 61 5 18 47 70 6 8 55 13 76 77 92
Quick sorted:
1 2 2 5 5 5 6 8 8 8 10 12 13 13 13 18 19 19 20 24 26 26 29 31 32 32 33 35 35 35 36 36 38 41 42 42 43 45 46 47 47 47 50 50 50
 52 52 52 53 53 53 53 54 55 55 55 58 60 61 61 62 64 65 66 70 73 73 75 76 77 77 79 79 81 81 81 83 85 85 86 86 88 88 88 88 90
90 92 92 93 93 94 95 96 96 97 97 98 98 99
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

- YES'

```cpp
template<typename T>
int partitionCustom(T arr[], int left, int right){
    T pivot = arr[right]; //gets pivot value
    int i = left - 1; //i to -1 or before first element of unsorted part of array

    for (int j = left; j < right; j++) {//go through array
        if (arr[j] <= pivot) { //if j < pivot,
            i++; //increment,
            std::swap(arr[i], arr[j]); //then swap(i,j)
        }
    }
    std::swap(arr[i + 1], arr[right]); //after j reaches pivot, swap pivot to i+1th element
    return ++i; //give the pivot's position
}

template<typename T>
void quickSorterCustom(T arr[], int left, int right) {
    if(left < right) {
        int pivotIndex = partitionCustom(arr, left, right); //first gets the pivotIndex's proper place
        int leftSubArray = pivotIndex - left;
        int rightSubArray = right - pivotIndex;
        if(leftSubArray > 1) {
            insertionSort(arr + left, leftSubArray);
        }
        if(rightSubArray > 1) {
            insertionSort(arr + (pivotIndex + 1), rightSubArray);
        }

    }
}

template<typename T>
void quickSortCustom(T arr[], size_t arrSize) {
   if(arrSize <= 1) {
        std::cout << "Array is already sorted.\n";
    } else {
        quickSorterCustom(arr, 0, arrSize - 1);
    }
}
```

```
Unsorted:
67 33 9 25 6 81 3 16 73 61 86 55 35 70 18 79 73 33 35 50 99 39 88 28 18 54 48 22 97 34 43 60 12 71 15 42 92 89 12 62 75 33
17 99 75 0 75 45 87 6 48 39 77 49 29 9 90 89 72 80 62 78 77 62 67 20 17 10 93 38 79 76 11 87 39 97 80 44 32 61 11 90 7 69 4
7 27 84 19 59 11 83 59 1 45 35 79 26 69 41 47
Quick sorted(Custom):
0 1 3 6 6 7 9 9 10 11 11 11 12 12 15 16 17 17 18 18 19 20 22 25 26 27 28 29 32 33 33 33 34 35 35 35 38 39 39 39 41 42 43 44
 45 45 47 47 48 48 49 50 54 55 59 59 60 61 61 62 62 62 67 67 69 69 70 71 72 73 73 75 75 75 76 77 77 78 79 79 79 80 80 81 83
 84 86 87 87 88 89 89 90 90 92 93 97 97 99 99
```

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity

- Merge sort and quick sort will have the best time complexity in sorting the array. Merge sort always has O(N log N) as its worst and best case. Quick sort will have near O(N log N) since the pivots are not in the worst places they could be(always in the edge of the array)

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Studying the process to make the functions**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The idea of sorting was already taught to us by our instructor and I only needed help in translating some parts into code.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.